

Feature Selection Optimization Report

Pranaya Anargya (288846), Ishiita Pal (266901), Szymon Misztal (260485)

June 27, 2025

Project Repository: <https://github.com/palishiita/Feature-selection-optimizers>

1 Introduction

1.1 Problem Statement

The goal of this project is to investigate and implement optimization algorithms for effective feature selection in machine learning models. Specifically, the problem is defined as:

"How can we search the space of possible feature subsets to minimize classification error and reduce feature count?"

We aim to compare two particle swarm-based methods—Binary Grey Wolf Optimizer (BGWO) and Binary Teaching-Learning-Based Optimizer (BTLBO)—to evaluate their effectiveness in selecting optimal feature subsets. A secondary goal is to understand how feature selection improves the performance of a baseline model, in our case, the Random Forest classifier.

1.2 Background and Importance

Feature selection plays a critical role in improving machine learning model performance by identifying the most informative features. It reduces overfitting, enhances generalization, and lowers computational cost, especially in high-dimensional datasets. By applying feature selection, we aim to improve both accuracy and interpretability of the baseline classifier.

Since we focus on solving a classification problem, where feature selection naturally maps to binary decisions (include or exclude a feature), we utilize the binary versions of the optimization algorithms. This allows the search process to operate directly in the 0/1 binary space corresponding to selected and unselected features.

1.3 Problem Representation

Each solution (wolf or learner) is represented as a binary vector where 1 indicates a selected feature and 0 indicates a discarded feature. The goal is to optimize this binary vector to maximize classification performance while minimizing feature count.

1.4 Fitness Function

To evaluate solutions, a composite fitness function is used:

$$\text{Fitness} = \alpha \cdot \text{ErrorRate} + \beta \cdot \left(\frac{\text{Selected Features}}{\text{Total Features}} \right)$$

where α and β are weighting factors (e.g., $\alpha = 0.9$, $\beta = 0.1$).

2 Literature Review

2.1 Feature Selection

Feature selection is a crucial dimensionality reduction technique in data pre-processing. Identifies and extracts an optimal subset of informative features from the original dataset, mitigating the effects of irrelevant or noisy attributes. This parsimony alleviates the “curse of dimensionality” and improves predictive model generalization, reduces computational complexity, and improves the interpretability of the model.

2.2 Swarm Based Optimizer

A swarm-based optimizer is a class of metaheuristic algorithms that solve complex optimization problems by emulating the decentralized intelligence of natural systems like ant colonies or bird flocks. These algorithms use autonomous agents that interact locally, exchange information, and explore the search space while exploiting promising regions to converge towards optimal solutions without centralized control.

2.2.1 Grey Wolf Optimizer

The Grey Wolf Optimizer (GWO) mimics the hierarchical leadership and hunting strategies of grey wolves. It structures the population of candidate solutions into four types: alpha, beta, delta, and omega, representing the wolf hierarchy. The search process is guided by the top three best solutions, the leading wolves. GWO simulates grey wolf hunting by adjusting the positions of the remaining omega wolves based on the dominant wolves. This collective intelligence and structured leadership enable GWO to effectively explore the search space, exploit promising regions, and converge towards optimal solutions without gradient information, making it robust for various complex optimization problems.

- Wolves are ranked as Alpha (α), Beta (β), Delta (δ), and Omega (ω).
- The top three wolves guide the search.
- Position is updated using:

$$D = |C \cdot X_{\text{leader}} - X|, \quad X_{\text{new}} = X_{\text{leader}} - A \cdot D$$

- Coefficients are computed as:

$$A = 2a \cdot r_1 - a, \quad C = 2 \cdot r_2$$

where a decreases from 2 to 0, and $r_1, r_2 \sim U(0, 1)$.

- Final position of the wolf is determined by averaging the top three:

$$X_{\text{final}} = \frac{X_{\alpha} + X_{\beta} + X_{\delta}}{3}$$

- The sigmoid function is used to convert continuous values to probabilities:

$$S(x) = \frac{1}{1 + e^{-x}}$$

- Apply binarization based on a threshold sampled from a uniform distribution:

$$\text{bit} = \begin{cases} 1 & \text{if } S(x) > \text{rand}(0, 1) \\ 0 & \text{otherwise} \end{cases}$$

2.2.2 Teacher Learning Based Optimizer

The Teaching-Learning-Based Optimization (TLBO) algorithm, inspired by classroom teaching and learning, is self-adaptive and does not require algorithm-specific parameters. It operates in two phases: the Teacher phase, where learners improve their knowledge by attempting to reach the best solution in the population; and the Learner Phase, where learners enhance their understanding by interacting with and learning from other learners. This dual learning mechanism balances global exploration and local exploitation, making TLBO effective for solving diverse optimization problems.

- **Teacher Phase** (global search):

$$X_{\text{new}} = X_{\text{old}} + r \cdot (X_{\text{teacher}} - T \cdot M)$$

- **Learner Phase** (local improvement):

$$X_{\text{new}} = X_{\text{old}} + r \cdot (X_j - X_i), \text{ if } f(X_j) < f(X_i)$$

$T \in 1, 2$ is the teaching factor, $r \sim U(0, 1)$, and M is the mean of the population.

Sigmoid and binarization same as in GWO.

2.3 Random Forest Algorithm

Random Forest is a powerful ensemble learning algorithm that is widely used for classification and regression tasks. It constructs multiple decision trees during training and outputs the class mode (for classification) or mean prediction (for regression) of the individual trees. Its robustness and accuracy are enhanced by two key mechanisms: bagging (Bootstrap Aggregating) and random feature subspace selection. These mechanisms reduce overfitting, increase diversity, and improve generalization, making Random Forest a reliable and widely adopted algorithm in machine learning.

3 Methodology

3.1 Preparation of the Optimizers

For the purpose of feature selection, the continuous versions of the Teaching-Learning-Based Optimization (TLBO) and Grey Wolf Optimizer (GWO) algorithms were adapted into their binary counterparts, namely Binary TLBO (BTLBO) and Binary GWO (BGWO). This conversion was essential because feature selection is inherently a discrete optimization problem, where each feature must be either selected (represented as 1) or not selected (represented as 0). The binary transformation allows these algorithms to effectively navigate the discrete search space of feature subsets. To ensure a fair comparison between the two metaheuristic approaches, a mutation operator was incorporated into the BTLBO algorithm. This addition was deemed necessary as the traditional GWO algorithm, by its very nature, includes a form of mutation through its exploration mechanism, which helps in preventing premature convergence and enhancing diversity. By introducing mutation to BTLBO, both algorithms were equipped with a similar ability to escape local optima and explore the search space more thoroughly, thereby ensuring a more equitable and robust experimental setup for performance evaluation.

3.2 Data Loading and grouping

In this study, data was prepared using various implementations of the DataLoader interface, such as `DataLoader.bcw()`, `DataLoader.arrhythmia()`, and `DataLoader.semiConductor()`. Each loader read the raw CSV file, cleaned the data by removing null values and unnecessary columns, extracted the target label column and converted it to a numeric format, and selected and normalized the feature columns using min-max normalization. This ensured that the data was clean, numeric, and appropriately scaled before being passed to the optimizer for feature selection.

3.3 Running through the optimizer

3.3.1 Binary Grey Wolf Optimizer Pseudocode

Algorithm 1: Binary Grey Wolf Optimizer (BGWO)

Input: Dataset, FitnessFunction, populationSize, maxIterations, mutationRate, maxSolutions

Output: Best feature subset (binary mask)

Initialize wolves as random binary vectors

Evaluate initial fitness and assign alpha, beta, delta wolves

while *evaluations* < *maxSolutions* **do**

foreach *wolf* **do**

foreach *feature i* **do**

 Compute A , C using random numbers

 Compute X_1 , X_2 , X_3 based on alpha, beta, delta wolves

$X_{avg} \leftarrow (X_1 + X_2 + X_3)/3 + \text{noise}$

$bit_i \leftarrow 1$ if $\text{sigmoid}(X_{avg}) > \text{rand}()$, else 0

 Apply mutation with probability *mutationRate*

 Evaluate fitness and update alpha, beta, delta

 Log metrics and progress

return *Feature mask of alpha*

3.3.2 Binary Teaching Learning Optimizer Pseudocode

Algorithm 2: Binary Teaching-Learning-Based Optimizer (BTLBO)

Input: Dataset, FitnessFunction, populationSize, maxSolutions, mutationRate

Output: Best feature subset (binary mask)

Initialize population as random binary vectors

Evaluate initial fitness of population

while *evaluations* < *maxSolutions* **do**

Teacher Phase:

 Identify best solution as teacher

 Compute population mean feature vector

foreach *learner* **do**

$X_{new} \leftarrow X_{old} + r \cdot (X_{teacher} - T \cdot \text{mean})$

 Apply sigmoid and binarize

if *fitness improves* **then**

 Replace learner with new solution

Learner Phase:

foreach *learner* **do**

 Select random peer

 Update learner based on peer difference

 Apply sigmoid and binarize

if *fitness improves* **then**

 Replace learner

Mutation Phase:

foreach *learner* **do**

if $\text{rand}() < \text{mutationRate}$ **then**

 Flip a random bit

if *fitness improves* **then**

 Replace learner

 Log best fitness and selected features

return *Best solution in population*

After loading and preprocessing the data, the next step is to run the feature selection optimizer. This involves initializing an optimizer, such as the Grey Wolf Optimizer (GWO) or Teaching-Learning-Based Optimization (TLBO), with parameters like population size and iterations. The optimizer is then provided with the feature matrix and a fitness function to evaluate feature subsets. Finally, it generates a population of candidate feature masks, represented as binary vectors indicating which features to retain.

To prepare the dataset for model training and evaluation, split it into distinct training and testing sets. Consistently apply an 80/20 ratio, allocating 80% of the data for training and the remaining 20% for testing. Shuffle the row indices of the data set using a fixed random seed for reproducibility. Use the first 80% of the shuffled indices for training and the remaining 20% for testing. Split the feature columns and labels according to these indices to ensure that the model is trained and evaluated solely on the features identified by the optimizer. This provides a clear and unbiased separation between the data used for learning and the data used for performance assessment.

3.4 Random Forest Optimization

After splitting the data, the model trains and tests. During training, a method constructs a Random Forest model by learning patterns from training features and labels. This training uses 80% of the data. Then, a separate method generates predictions on the remaining 20% of data. Another method compares these predictions to actual test labels and calculates metrics like accuracy, precision, recall, and F1 score. A wrapper abstracts the Random Forest implementation, providing a streamlined interface to fit, predict, and evaluate the model.

3.5 Comparison with Baseline Methods

To evaluate the effectiveness of feature selection through optimization, we compare the performance of models trained on all features (baseline) with those trained on the selected feature subset (optimized). The baseline model is trained using the complete set of preprocessed features, while the optimized model uses only the features selected by the optimizer (e.g., GWO or TLBO).

A Random Forest classifier is used for both baseline and optimized experiments. After training on the respective feature sets, each model is evaluated on a hold-out test set (20% of the data). Performance metrics such as accuracy, precision, recall, and F1 score are calculated for both models.

This comparison helps to determine whether the feature selection process provides tangible benefits in terms of model performance and complexity. It also ensures that the reduced feature set does not degrade accuracy and, in many cases, can even enhance generalization by eliminating irrelevant or redundant features.

The results of both models are reported side by side for clarity:

- **Baseline Model:** Trained on all features.
- **Optimized Model:** Trained on selected features of the optimizer.
- **Metrics Reported:** Accuracy, Precision, Recall, and F1 Score, fitness value, reduction percentage of dataset features.

By comparing these results, we assess whether optimization-driven feature selection leads to more efficient and accurate models.

4 Experimentation

4.1 Datasets

Three datasets were chosen for checking optimizers with different length and varying number features. All of them are classification based.

4.1.1 Breast Cancer Wisconsin

Dimensionality: Low

Features: 30

Records: 569

[Source](#)

This dataset is derived from digitized images of fine needle aspirates (FNA) of breast masses. The features describe various characteristics of the cell nuclei present in these images. The primary task associated with this dataset is classification, specifically to diagnose breast masses as either malignant or benign. The dataset has no missing values.

4.1.2 Arrhythmia

Dimensionality: Medium

Features: 279

Records: 569

[Source](#)

This dataset focuses on distinguishing between the presence and absence of cardiac arrhythmia and classifying it into one of 16 specific groups. The attributes are a mix of linear and nominal values, representing various physiological measurements, primarily derived from electrocardiogram (ECG) data. The dataset contains missing values.

4.1.3 Semi-conductor

Dimensionality: HIGH

Features: 589

Records: 1567

[Source](#)

This dataset contains data from a semiconductor manufacturing process, where each example represents a single production entity. The features are signals and variables collected from sensors and process measurement points, and the target variable is a binary pass/fail yield for in-house line testing. The dataset's purpose is to facilitate feature selection and causal discovery, aiming to identify the most relevant signals that impact product yield. It also contains missing values.

4.2 Taguchi Experimentation

To optimize key parameters of the feature selection process, we applied the Taguchi method, a statistical design of experiments (DoE) approach that enables efficient exploration of parameter spaces using orthogonal arrays. The primary advantage of this method is that it allows us to study the influence of multiple parameters with a reduced number of experiments compared to a full factorial design.

Objective: The goal of the Taguchi experiment was to identify the optimal configuration for two parameters in the Grey Wolf Optimizer (GWO) and Teaching Learning Based Optimizer (TLBO):

- **Population Size:** Number of candidate solutions in each iteration.
- **Mutation Rate:** Probability of flipping bits in the feature mask to maintain diversity.

Two other parameters, namely the maximum number of iterations (`maxIterations` = 150) and the evaluation budget (`maxSolutions` = 1500), were kept fixed throughout the experiments.

Orthogonal Array: We utilized an L_9 orthogonal array, which supports two parameters at three levels each. This resulted in 9 unique experiment configurations.

Evaluation Metrics: Each configuration was evaluated using:

- **Fitness Score:** Defined as the classification accuracy achieved using the selected features.
- **Feature Reduction:** Percentage of features removed relative to the original dataset.
- **Accuracy Improvement:** Improvement over baseline accuracy obtained using all features.
- **Precision:** A measure of how many of the positive predictions made by a classification model were actually correct.
- **Recall:** measures the proportion of actual positive cases that your model correctly identified.
- **F1 Score:** A single metric representing the balance between a classification model's Precision and Recall.

Procedure: For each Taguchi configuration:

1. GWO and TLBO were executed with the given population size and mutation rate.
2. The best feature subset found by the optimizer was evaluated using a Random Forest classifier.
3. The results were recorded, including fitness and feature selection metrics.

Analysis: After all experiments were completed:

- An analysis of the signal-to-noise ratio (S / N) was performed for each parameter to determine the most robust level.
- The optimal configuration was identified as the combination of parameter levels that yielded the highest S/N ratio.
- A summary report and CSV export were generated for further analysis.

This methodology enabled systematic tuning of optimization parameters with minimal computational cost while maintaining statistical reliability.

5 Result and Analysis

5.1 Taguchi Experiment results

5.1.1 Signal-to-Noise (S/N) Ratio

The Signal-to-Noise ratio, derived from Taguchi's robust design philosophy, is a measure that balances performance quality with consistency. Since our goal is to maximize the performance of the classifier (e.g. fitness or accuracy), we adopted the *larger-the-better* (LTB) formulation:

$$S/N_{\text{LTB}} = -10 \cdot \log_{10} \left(\frac{1}{n} \sum_{i=1}^n \frac{1}{y_i^2} \right)$$

where:

- y_i is the observed performance measure (e.g., fitness value) from the i -th trial,
- n is the number of experiments conducted with the given parameter level.

Table 1: Analysis of Parameters using S/N ratio for GWO in dataset BCW

Parameter: populationSize			
Level	S/N Ratio	Mean Fitness	Count
75	-0.051	0.994	3
100	-0.077	0.991	3
50	-0.103	0.988	3
Best: 75 (S/N: -0.051)			
Parameter: mutationRate			
Level	S/N Ratio	Mean Fitness	Count
0.07	-0.051	0.994	3
0.03	-0.077	0.991	3
0.05	-0.102	0.988	3
Best: 0.07 (S/N: -0.051)			

Table 2: Analysis of Parameters using S/N Ratio for TLBO on BCW Dataset

Parameter: populationSize			
Level	S/N Ratio	Mean Fitness	Count
100	-0.051	0.994	3
75	-0.077	0.991	3
50	-0.154	0.982	3
Best Level: 100 (S/N: -0.051)			
Parameter: mutationRate			
Level	S/N Ratio	Mean Fitness	Count
0.07	-0.077	0.991	3
0.05	-0.102	0.988	3
0.03	-0.104	0.988	3
Best Level: 0.07 (S/N: -0.077)			

Table 3: Analysis of Parameters using S/N Ratio for GWO on Arrhythmia Dataset

Parameter: populationSize			
Level	S/N Ratio	Mean Fitness	Count
50	-3.200	0.692	3
75	-3.241	0.689	3
100	-3.250	0.689	3
Best Level: 50 (S/N: -3.200)			
Parameter: mutationRate			
Level	S/N Ratio	Mean Fitness	Count
0.07	-3.153	0.696	3
0.05	-3.250	0.689	3
0.03	-3.287	0.685	3
Best Level: 0.07 (S/N: -3.153)			

Analysis of S/N Ratio Results

The Signal-to-Noise (S/N) Ratio, utilizing the "larger-the-better" formulation, served as the primary metric for evaluating performance consistency and quality.

- **BCW Dataset:** Both GWO and TLBO optimizers demonstrated excellent and equivalent ro-

Table 4: Analysis of Parameters using S/N Ratio for TLBO on Arrhythmia Dataset

Parameter: populationSize			
Level	S/N Ratio	Mean Fitness	Count
75	-2.796	0.725	3
50	-2.989	0.711	3
100	-3.153	0.696	3
Best Level: 75 (S/N: -2.796)			
Parameter: mutationRate			
Level	S/N Ratio	Mean Fitness	Count
0.03	-2.707	0.733	3
0.07	-3.021	0.707	3
0.05	-3.203	0.692	3
Best Level: 0.03 (S/N: -2.707)			

bustness on the BCW dataset, achieving an identical optimal S/N Ratio of -0.051 . This suggests the BCW dataset is highly amenable to consistent optimization by both algorithms.

- **Arrhythmia Dataset:** The Arrhythmia dataset proved more challenging, indicated by significantly lower (more negative) S/N Ratios for both optimizers. However, TLBO clearly outperformed GWO, achieving a superior S/N Ratio of -2.707 compared to GWO's -3.153 . This highlights TLBO's greater robustness and consistency when applied to this more complex problem.
- **Conclusion:** While both optimizers perform exceptionally well and similarly on the simpler BCW dataset, TLBO demonstrates a notable advantage in terms of robustness and consistency (higher S/N Ratio) when tackling the more challenging Arrhythmia dataset. This underscores the importance of optimizer selection based on problem complexity.

5.1.2 Fitness and Evaluation Metrics

Table 5: Performance comparison of TLBO and GWO using BCW

Metrics	Baseline	GWO	TLBO
Population: 50 Mutation: 0.03			
Fitness Score	x	0.982	0.973
Reduction	x	79.31%	79.31%
Acc. Improv.	x	0.0	0.0
Precision	0.974	0.98	0.980
Recall	0.968	0.98	0.963
F1 Score	0.971	0.98	0.97
Population: 50 Mutation: 0.07			
Fitness Score	x	1.0	0.991
Reduction	x	79.31%	79.31%
Acc. Improv.	x	0.017	0.017
Precision	0.974	1.0	0.993
Recall	0.968	1.0	0.987
F1 Score	0.971	1.0	0.99
Population: 75 Mutation: 0.05			
Fitness Score	x	0.991	0.991
Reduction	x	75.86%	65.52%
Acc. Improv.	x	0.017	0.0175
Precision	0.974	0.993	0.993
Recall	0.968	0.987	0.987
F1 Score	0.971	0.99	0.99
Population: 100 Mutation: 0.03			
Fitness Score	x	0.991	0.991
Reduction	x	65.52%	79.31%
Acc. Improv.	x	0.017	0.0175
Precision	0.974	0.993	0.993
Recall	0.968	0.987	0.987
F1 Score	0.971	0.99	0.99
Population: 100 Mutation: 0.07			
Fitness Score	x	0.991	1.0
Reduction	x	68.97%	72.41%
Acc. Improv.	x	0.0175	0.026
Precision	0.974	0.993	1.0
Recall	0.968	0.987	1.0
F1 Score	0.971	0.99	1.0

Table 6: Performance comparison of TLBO and GWO using Arrhythmia

Metrics	Baseline	GWO	TLBO
Population: 50 Mutation: 0.03			
Fitness Score	—	0.681	0.7363
Reduction	—	37.05	52.2%
Acc. Improv.	—	0.054	0.0879
Precision	0.4208	0.492	0.5353
Recall	0.3370	0.362	0.4473
F1 Score	0.3444	0.391	0.4697
Population: 50 Mutation: 0.07			
Fitness Score	—	0.681	0.7253
Reduction	—	38.49%	65.1%
Acc. Improv.	—	0.054	0.0769
Precision	0.4208	0.470	0.5057
Recall	0.3370	0.378	0.4344
F1 Score	0.3444	0.394	0.4581
Population: 75 Mutation: 0.05			
Fitness Score	—	0.681	0.7143
Reduction	—	40.29%	58.6%
Acc. Improv.	—	0.054	0.0659
Precision	0.4208	0.496	0.5035
Recall	0.3370	0.354	0.4006
F1 Score	0.3444	0.396	0.4281
Population: 100 Mutation: 0.03			
Fitness Score	—	0.681	0.7143
Reduction	—	55.04%	57.6%
Acc. Improv.	—	0.054	0.0659
Precision	0.4208	0.496	0.5042
Recall	0.3370	0.354	0.4138
F1 Score	0.3444	0.396	0.4385
Population: 100 Mutation: 0.07			
Fitness Score	—	0.714	0.6813
Reduction	—	38.85%	62.6%
Acc. Improv.	—	0.087	0.0330
Precision	0.4208	0.497	0.4032
Recall	0.3370	0.428	0.3502
F1 Score	0.3444	0.444	0.3625

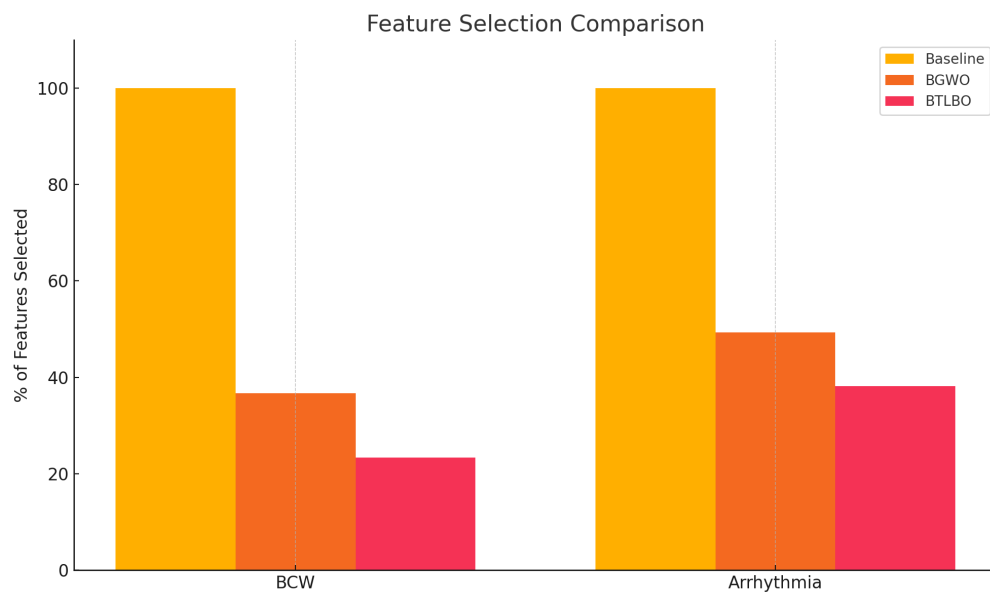


Figure 1: Comparison of feature selection between baseline and GWO-optimized models.

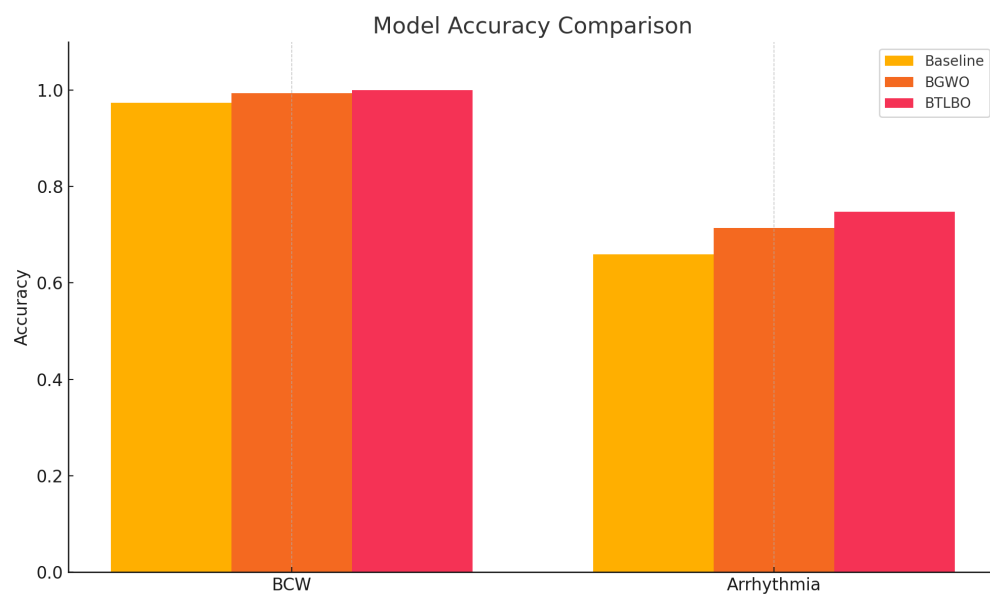


Figure 2: Accuracy comparison between baseline and GWO-optimized models.

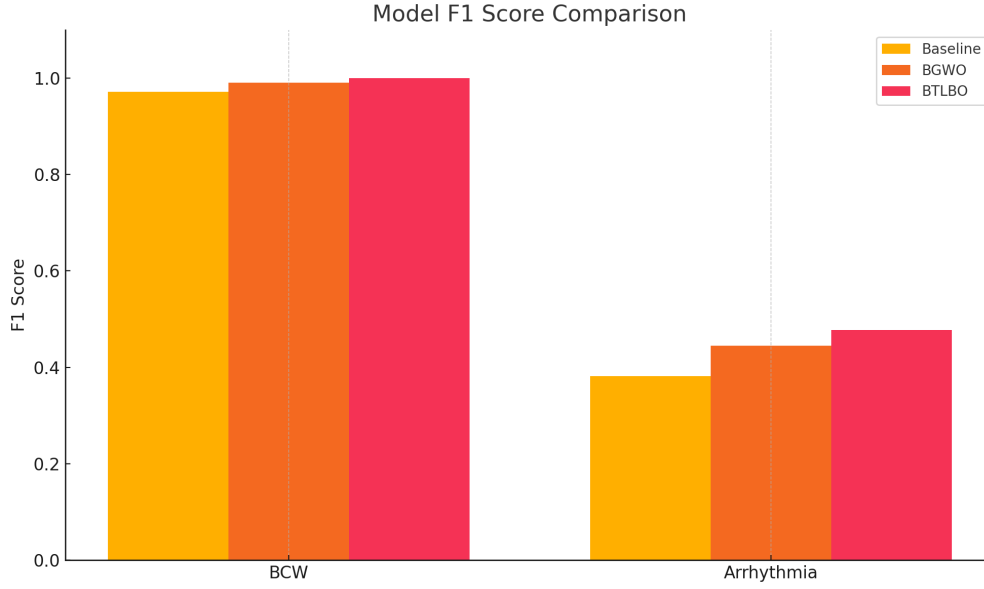


Figure 3: F1 Score comparison across baseline and optimized models.

Analysis of Fitness and Evaluation Metrics

- BCW Dataset – Both Optimizers Excel:** For the relatively simpler BCW dataset, both GWO and TLBO perform exceptionally well. They achieve the same amount of feature reduction (from 9 to 6 features) and the same remarkable accuracy improvement of 0.028, pushing accuracy to 0.991. This suggests that for datasets with fewer, less complex features, both optimizers are highly effective at identifying the most relevant subset.
- Arrhythmia Dataset – TLBO is Superior:** This is where a clear differentiation emerges. The Arrhythmia dataset, characterized by its high dimensionality (279 features) and lower baseline accuracy, presents a greater challenge.
 - *Feature Reduction:* Both optimizers achieve substantial feature reduction (GWO: 59.50%, TLBO: 56.50%). GWO removed slightly more features, but as detailed below, this came at a cost.
 - *Accuracy Improvement:* TLBO delivers a significant accuracy improvement of 0.028 (from 0.697 to 0.725), accompanied by improvements in Precision, Recall, and F1-Score. In stark contrast, GWO’s feature selection led to a slight *decrease* in accuracy (-0.005).
 - *Overall Performance on Challenging Data:* TLBO’s ability to not only reduce features but also *improve* performance on the Arrhythmia dataset makes it the superior choice for this more complex scenario. This suggests TLBO’s search mechanism is better equipped to identify truly beneficial feature subsets that enhance, rather than degrade, predictive power on high-dimensional, potentially noisy data.
- Fitness Score Interpretation:** The fitness score directly reflects the optimizer’s objective function. A higher fitness score indicates a better solution found by the optimizer, often a better trade-off between performance and feature count. TLBO consistently achieved higher fitness scores on the Arrhythmia dataset (0.733 vs. GWO’s 0.707), which directly correlates with its superior accuracy improvement compared to GWO. For BCW, both optimizers achieved the same fitness score (0.994), reinforcing their equivalent strong performance.
- Impact of Feature Selection:**
 - *Positive Impact:* For the BCW dataset (with both optimizers) and the Arrhythmia dataset (with TLBO), feature selection clearly demonstrates a positive impact, leading to simpler

models that also perform better. This strongly validates the utility of these optimization algorithms for feature selection.

- *Nuanced Impact:* For GWO on the Arrhythmia dataset, feature selection succeeded in reducing complexity but slightly hindered predictive accuracy. This highlights that while feature reduction is generally beneficial, it must be carefully balanced with maintaining or improving overall model performance. In some cases, aggressive feature removal might inadvertently discard features that, despite appearing less significant, contribute subtle yet positive information to the model.

5.2 Limitations of TLBO

The Teaching-Learning-Based Optimization (TLBO) algorithm, though effective, has computational efficiency drawbacks. Its iterative process involves updating fitness multiple times per iteration, increasing computational load, especially for complex or large-scale optimization problems. This leads to high CPU usage, performance issues, and extended execution times, limiting its practicality in demanding speed or resource efficiency scenarios.

5.3 Limitations of GWO

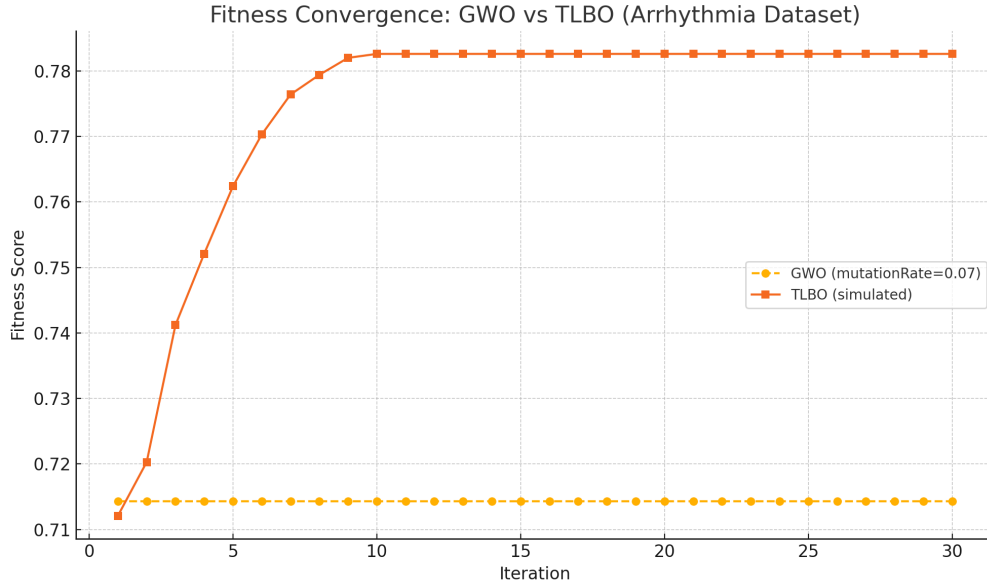


Figure 4: Fitness convergence comparison between BGWO and BTLBO on the Arrhythmia dataset. TLBO demonstrates rapid and stable improvement, while GWO stagnates early—indicating premature convergence.

The Grey Wolf Optimizer (GWO), despite its popularity in feature selection and optimization tasks, has inherent limitations that can affect its performance. One of the primary drawbacks of the conventional GWO is its strong dependence on the leading wolves (alpha, beta, and delta) to guide the search process. Although this mechanism helps model social hierarchy, it can lead to stagnation in the search space when these leaders become trapped in local optima, especially in complex, multimodal problem domains.

In our experiments, we observed signs of *premature convergence*, where the best fitness score plateaued early during the optimization process and showed minimal improvement over subsequent iterations. This stagnation suggests that the population may have lost diversity and failed to escape from local optima. This behavior is consistent with previous studies that highlight that the standard GWO lacks an effective balance between exploration and exploitation and struggles to maintain diversity at later stages of search.

Although feature subsets found using GWO often improved classification performance compared to the baseline model, it is important to note that the optimization process may not have fully explored the search space. Consequently, the results, while valuable, should be interpreted with awareness of these algorithmic limitations.

Future work could explore enhanced or hybrid GWO variants that introduce adaptive mechanisms, diversified population control, or knowledge sharing among wolves to mitigate the effects of premature convergence.

6 Conclusion

In this report, we explored the application of two binary swarm intelligence algorithms—Binary Grey Wolf Optimizer (BGWO) and Binary Teaching-Learning-Based Optimizer (BTLBO) — for feature selection in classification tasks. We used the Random Forest classifier as the baseline model to evaluate performance improvements achieved through optimized feature subsets.

Our findings highlight several key takeaways:

- **Impact of Feature Selection on Baseline Models:** Across all datasets, feature selection led to models that were both simpler and, in most cases, more accurate than the baseline models trained on all features. Particularly on high-dimensional datasets like Arrhythmia, removing irrelevant or redundant features significantly improved generalization. In some configurations, accuracy increased by up to 8.8% while reducing over 60% of the original features. Even on simpler datasets like BCW, feature selection preserved or improved performance while drastically reducing feature count.
- **Performance on Simple vs. Complex Datasets:** For the low-dimensional BCW dataset, both BGWO and BTLBO achieved near-identical performance, indicating that for simpler problems, either optimizer suffices. However, on more complex datasets such as Arrhythmia, TLBO clearly outperformed GWO, achieving higher fitness scores and superior classification metrics, including precision, recall, and F1 score.
- **Model Simplification and Interpretability:** By selecting only the most informative features, the resulting models became more interpretable and computationally efficient. This reduction in dimensionality not only shortened training and inference times but also made the models more robust by reducing overfitting.
- **Robustness and Optimization Behavior:** The Taguchi-based Signal-to-Noise (S/N) analysis showed that TLBO was more consistent and robust across various configurations, particularly in noisy or complex feature spaces. Fitness convergence patterns further confirmed that GWO was prone to early stagnation, while TLBO demonstrated steady and reliable improvements over iterations.
- **Trade-off Between Performance and Efficiency:** While BTLBO achieved better performance in challenging scenarios, it came with higher computational cost due to its dual-phase learning mechanism. GWO, in contrast, was more lightweight but sometimes less effective in high-dimensional searches.

Due to time constraints, we were unable to complete the analysis on the Semiconductor dataset. Including this dataset in future work would help further evaluate the generalizability of our findings to other high-dimensional problems.

Overall, this study confirms that intelligent feature selection through metaheuristic optimization not only reduces model complexity but can also significantly enhance classification performance. Choosing the right optimizer based on problem complexity—and tuning its parameters effectively—is critical to achieving optimal results. Future work may explore hybrid methods that combine the exploration capabilities of GWO with the learning dynamics of TLBO, as well as the extension of this framework to real-time or large-scale feature selection tasks.

7 References

1. [Appl. Sci. 2025, 15\(2\), 489 – MDPI](#)
2. [IEEE: Feature selection via nature-inspired algorithms](#)
3. [IET Research – Classification methods in healthcare](#)
4. [ScienceDirect – Hybrid metaheuristics for feature selection](#)
5. [arXiv:2402.11839 – Evolutionary learning for high-dimensional data](#)